# The HPC@PoliTO User Guide

July 2026, v2

# Contents

**Basic Rules** **7**

**1 The HPC@PoliTO service** **9**
1.1 Account request . . . . . 9
1.2 Storage policies . . . . . 9

**2 First login to the clusters** **11**
2.1 Connecting to the Legion login node via *ssh* . . . . . 11
2.2 Using PuTTY (only on Windows machine) . . . . . 12
2.3 Connection via passwordless-ssh . . . . . 12
2.4 Copying files to and from the cluster . . . . . 13

**3 The HPC@PoliTO Infrastructure** **15**
3.1 The Legion cluster . . . . . 15
3.1.1 Legion Isola 1 . . . . . 15
3.1.2 Legion Isola 2 . . . . . 15
3.2 Partitions . . . . . 17
3.2.1 Open access partitions . . . . . 18
3.2.2 Restricted access partitions . . . . . 18
3.2.3 EDU partitions . . . . . 19
3.3 Cluster networking . . . . . 19
3.4 Storage systems . . . . . 21
3.4.1 Home storage . . . . . 21
3.4.2 BeeGFS scratch . . . . . 21
3.5 Overall architecture . . . . . 22

**4 HPC@PoliTO file systems** **23**
4.1 Home directories . . . . . 23
4.2 BeeGFS scratch: high performance storage . . . . . 23
4.3 File transfer between different file systems . . . . . 24
4.4 Best practices . . . . . 24

**5 Software ecosystem** **27**
5.1 Environment Modules . . . 27
5.2 Spack . . . 28
5.3 Apptainer . . . 29

**6 Jobs submission** **31**
6.1 Shared resources . . . 31
6.2 Submission script . . . 31
6.2.1 Job parameters . . . 32
6.2.2 Use the BeeGFS filesystem . . . 33
6.2.3 Simulation run . . . 33
6.3 Submit a job request . . . 34
6.4 Check the simulation status . . . 34
6.5 Submission script examples . . . 34
6.6 Interactive sessions . . . 37
6.6.1 Quick interactive shell with `srun` . . . 37
6.6.2 Resource allocation with `salloc` . . . 37

**7 GPU workflow** **39**
7.1 GPU resources overview . . . 39
7.2 Requesting GPUs in SLURM . . . 39
7.2.1 CPU and memory allocation . . . 40
7.3 GPU environment setup . . . 40
7.3.1 Loading CUDA . . . 40
7.3.2 Verifying GPU allocation . . . 40
7.4 Interactive GPU debugging . . . 41
7.5 Common GPU issues . . . 41

**A Basic Linux usage from command line (CLI)** **43**
A.1 Basic terminal commands . . . 43
A.1.1 `man` and `--help` command . . . 43
A.1.2 `pwd`: your location . . . 43
A.1.3 `ls`: See the element inside a directory . . . 43
A.1.4 `cd`: navigate inside the file system . . . 44
A.1.5 Creating and removing directories . . . 44
A.1.6 `Vim`: file editor . . . 44

**B Connection to the clusters via passwordless-ssh** **45**
B.1 Generate key pair from a terminal . . . 45
B.1.1 Copying the Public Key . . . 45
B.1.2 Login . . . 46
B.2 Using PuTTY . . . 47
B.2.1 Generating a Key with PuTTYgen . . . 47
B.2.2 Copying the Public Key . . . 47
B.2.3 Logging in with PuTTY . . . 48

**C The SLURM scheduler** **51**
C.1 Priority calculation . . . . . 52

**D Restricted partitions** **53**

# Basic Rules

1. **Never execute simulations or intensive tasks directly in your shell**: this would disrupt operations on the login server, making the HPC infrastructure unavailable for the whole community.

2. To execute their jobs, users should submit batch requests to the SLURM scheduler. For intensive post-processing, code compilation or large data transfers, users should instead request interactive sessions to the SLURM scheduler.

3. The HPC@PoliTO infrastructures should only be used for academic or teaching purposes.

4. Users are responsible for all activities originating from their account. They should therefore be aware about their account security, adopt safe practices and avoid accounts sharing.

5. Users agree to acknowledge the HPC@PoliTO initiative in any publications and talks that use data originating from our facilities. We suggest the following citation:

   "Computational resources provided by HPC@PoliTO (www.hpc.polito.it)"

6. After the expiration of an HPC account, data in the home directory are stored for a limited amount of time (t.b.d., hypothetically 6 months). Users are responsible for collecting and downloading their data before their account expires or asking HPC@PoliTO for their recovery as soon as the account expires (provided it will not be extended).

# Chapter 1

# The HPC@PoliTO service

The HPC@PoliTO service is hosted and managed by the Cybersecurity and IT for Research Division (ISIAD) at Politecnico di Torino. It provides high-performance computing resources for research and educational activities. The HPC@PoliTO infrastructure is made of several components, for our users the most relevant are: a compute cluster, a storage infrastructure and a login interface. Currently, the compute cluster of the HPC@PoliTO service is Legion. It was first deployed in 2019 and underwent a major upgrade at the end of 2024. Legion is used by many research groups at Politecnico di Torino (PoliTO), whose financial contributions are crucial for the overall evolution of the cluster.

Hactar, our legacy cluster (deployed in 2015), was decommissioned in 2025 due to aging hardware, increasing maintenance overhead and poor energy efficiency.

## 1.1 Account request

Access is granted to researchers, Ph.D. students and master students involved in their master thesis project. For the latters, requests must be submitted by their academic supervisor. Accounts are typically activated (or renewed) within 7 working days, although during workload peaks it may take longer. Access is also provided for educational purposes (classes) and for student teams; in these cases requests must be submitted by teachers or teams' academic supervisors.

Requests must be submitted via mail to `hpc@polito.it`.it attaching the relevant form, adequately filled, that can be found at the bottom of our webpage `hpc.polito.it`.

## 1.2 Storage policies

**Data storage** is currently **NOT GUARANTEED** whatsoever. Users are responsible for the safety of their data and are heavily encouraged to make their

own periodic backups. HPC@PoliTO provides several storage systems that users can choose according to their project needs. As will be explained later, some of these storage systems implement redundancy and resiliency mechanisms while others do not. These shall not be considered as backup measures whatsoever.

# Chapter 2

# First login to the clusters

## 2.1 Connecting to the Legion login node via *ssh*

The Legion HPC cluster can be used almost exclusively from a Linux terminal shell. If you are not familiar with Linux and its terminal, in Appendix A we reported some basic commands to get started with it. HPC@PoliTO cannot provide formal education on Linux and its usage.

To connect to the cluster, use the `ssh` protocol (Secure Shell). Open a new shell window in your terminal. On Linux and macOS devices, you can use the native terminal. For Windows users, we recommend using a PowerShell terminal, though it is also possible to open an `ssh` connection directly from the Command Prompt. The shell window can be opened in any directory.

The following command (where *username* is the one your received upon the activation of your account) opens an ssh session on the cluster:

```
ssh username@hpc-legionlogin.polito.it
```

If this is the first time you are connecting to the cluster, you will be asked to authenticate the machine, confirm by typing `yes`). Then you will be asked to prompt the One-Time Password (OTP) that you have been provided when the account was activated. The system will ask you immediately to change your password: it will ask again for the current password (the temporary password you received) and then to prompt (and confirm) the new one. Follow the given instructions!

> **Tip:** when entering the password, no characters or placeholders will appear on the screen. This is normal Linux behavior. Type your password and press `enter`.

Upon successful authentication, a new bash shell will be opened, the cluster's welcome page will appear and you will be in your home directory on the Home storage: `/home/username`.

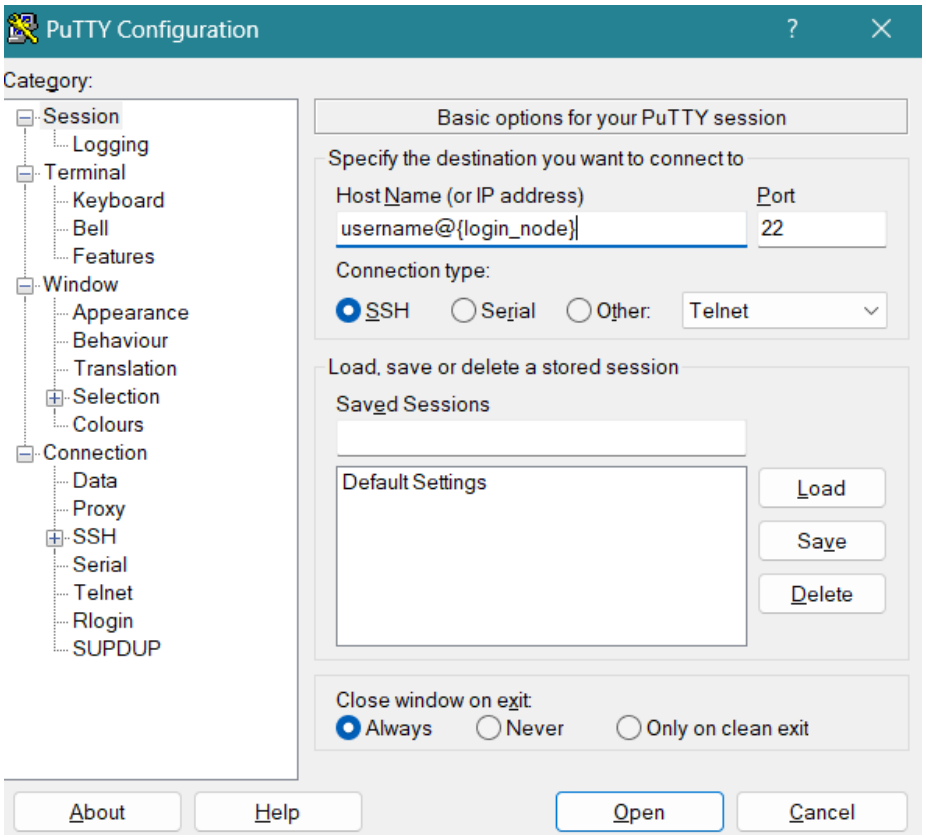



Figure 2.1: PuTTY setup

## 2.2 Using PuTTY (only on Windows machine)

From a Windows device, instead of using the terminal, it is possible to use PuTTY, a third-part application, to log in to the cluster. PuTTY emulates a Linux terminal on your host device and is available for download at `https://putty.org/`. To configure PuTTY, enter in the host name section your username and the login node to which you want to connect. We suggest to save the session using the specific area on the PuTTY main page. Once you have completed the procedure, open the connection, you will be prompted for the password.

## 2.3 Connection via passwordless-ssh

After the first login, users should set up passwordless SSH authentication using RSA key pairs as described in Appendix B.

## 2.4 Copying files to and from the cluster

If you need to copy files or directories between your local host and the cluster (or vice versa), you can use the `scp` command (Secure Copy Protocol). Open a new shell window on your local host and use the following commands: `scp` (Secure Copy Protocol). From your host to the cluster

```
scp -r /local_path username@hpc-legionlogin.polito.it:/path_server_side
```

From the cluster to your host (notice that the following command must be run on your host)

```
scp -r username@hpc-legionlogin.polito.it:/path_server_side /local_path
```

If you have not set up the passwordless login, you will be prompted your password, otherwise, the copying process starts. The option `-r` is needed when copying directories, as it enables recursive copying of all files and subdirectories.

# Chapter 3

# The HPC@PoliTO Infrastructure

## 3.1 The Legion cluster

Legion was originally deployed in 2019 and underwent a first early expansion in 2020. Between the end of 2024 and the first half of 2025, the cluster was the subject of a major upgrade that roughly doubled its compute node count. To manage different technologies between old and new hardware, two major partitions have been defined: Isola 1 and Isola 2.
Although Isola 1 and Isola 2 are logically separated and jobs should not be run across the two (to avoid bottlenecks due to different generations hardware working together), overall Legion is considered a single cluster: sharing login, management and storage infrastructures. When submitting jobs users should specify which partition they want to use.
Legion is a shared resource batch system whose job scheduling is based on SLURM (https://www.schedmd.com/slurm/). Further information about job submission and scheduling policies can be found in Chapter 6.

### 3.1.1 Legion Isola 1

Legion Isola 1 collects early-deployed compute nodes (from 2019-2020) which are then further partitioned accordingly to different access policies. Isola 1, as represented in Figure 3.1 contains 58 compute nodes, 6 of which are configured with NVIDIA V100 GPUs. The compute nodes within Isola 1 have the characteristics reported in Table 3.1.

### 3.1.2 Legion Isola 2

Legion Isola 2 was deployed between late 2024 and early 2025 following a dramatic expansion of computing resources thanks to the contribution of research

| Node type | CPU | Cores | RAM | GPU | Disk |
|---|---|---|---|---|---|
| CPU node | 2 x Intel Xeon 6130 | 32 | 384 GB | N/A | 1 TB HDD |
| GPU node | 2 x Intel Xeon 6130 | 32 | 384 GB | 4 x V100 (32 GB) | 1 TB HDD |

Table 3.1: Legion Isola 1 - Compute nodes

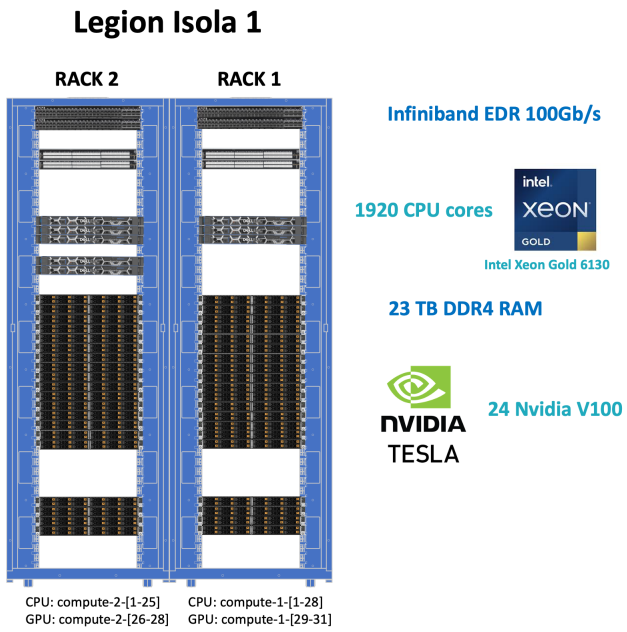



Figure 3.1: Legion Isola 1

| Node type | CPU | Cores | RAM | GPU | Disk |
|---|---|---|---|---|---|
| CPU node | 2 x Intel Xeon 6442Y | 48 | 512 GB | N/A | 1 TB NVMe |
| GPU node A40 | 2 x Intel Xeon 6442Y | 48 | 512 GB | 4 x A40 (48 GB) | 1 TB NVMe |
| GPU node A100 | 2 x Intel Xeon 6442Y | 48 | 1024 GB | 4 x A100 (80 GB) | 1 TB NVMe |
| GPU node H200 | 2 x Intel Xeon 8462Y | 64 | 2048 GB | 8 x H200 (141 GB) | 1 TB NVMe |

Table 3.2: Legion Isola 2 - Compute nodes

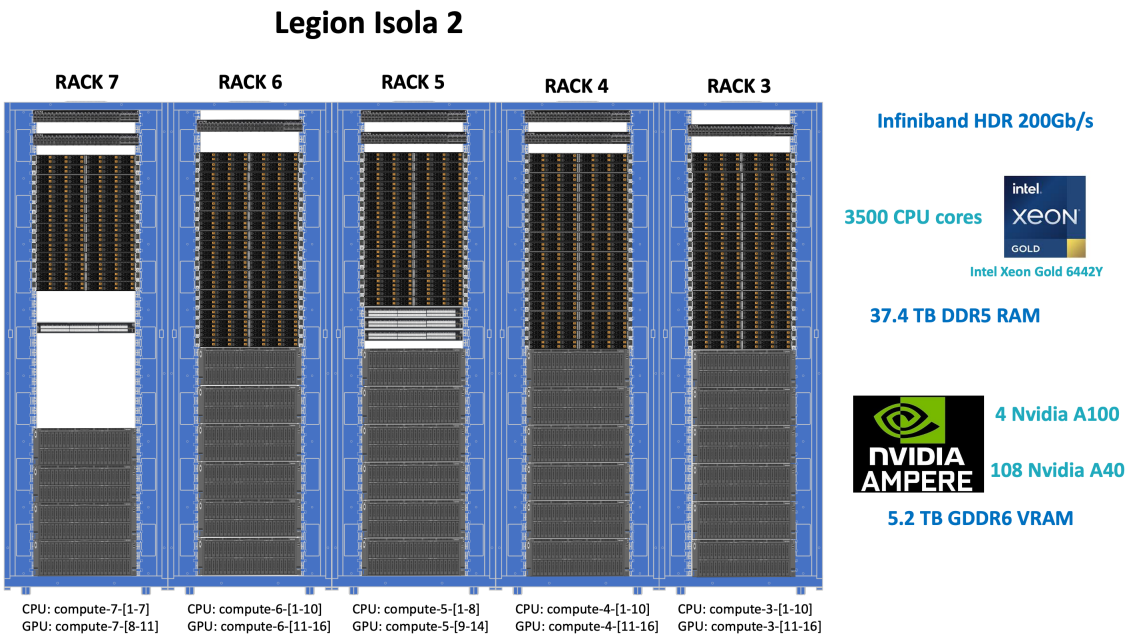



Figure 3.2: Legion Isola 2

groups at PoliTO and the availability of NextGenEU (PNRR) grants. Isola 2, as represented in Figure 3.2 contains 74 compute nodes, 26 of which are configured with NVIDIA A40 GPUs, 1 equipped with NVIDIA A100 GPUs and another one equipped with NVIDIA H200 GPUs. The compute nodes within Isola 2 have the characteristics reported in Table 3.2.

## 3.2 Partitions

The nodes within Isola 1 and Isola 2 are logically divided into partitions with different hardware and access policies.

Based on their scheduling policies, we defined two main kinds of partition:

**"Open access"** and **"Restricted access"**.

Then, based on the hardware available on their compute nodes we defined the following families: **"skylake"** (based on the older Intel Skylake CPUs), **"sapphire"** (based on the newer Intel Sapphire Rapids CPUs), **"gpu_v100"**, **"gpu_a40"**, **"gpu_a100"**, **"gpu_h200"** (based respectively on V100, A40, A100 and H200 NVIDIA GPUs).

### 3.2.1 Open access partitions

Open access partitions are available to all HPC@PoliTO users, with the following conditions:

- **Uniform priority**: all jobs submitted to these partitions have the same base priority;
- **Limited walltime**: all jobs submitted to these partitions are subject to strict maximum walltimes (24 hours for newer partitions, longer for the older ones)

Open access partitions on Legion are reported in Table 3.3. Partitions ending with suffix "ext" have a longer maximum walltime.

### 3.2.2 Restricted access partitions

Restricted access partitions are available only to specific research groups (which have financially contributed to the infrastructure) and their collaborators, with the following conditions:

- **Highest priority**: all jobs submitted to these partitions are given the maximum base priority, to guarantee they will be scheduled immediately when the resources are available;
- **Unlimited walltime**: all jobs submitted to these partitions are not subject to any temporal restriction;
- **Guaranteed waiting time**: jobs scheduled on these partitions are guaranteed a maximum waiting time (further details follow)

Contrary to past versions, Legion does not implement job suspension or preemption: high priority jobs would eventually need to wait for jobs running on open partitions to finish before they can start. Given that open access partitions in Isola 1 and Isola 2 are subject to a maximum walltime (120 and 24 hours respectively), these are also the **maximum waiting times for jobs scheduled on restricted partitions** in Isola 1 and Isola 2. Restricted access partitions on Legion are reported in Appendix D Table D.1.

| Partition | N nodes | Hardware node | Priority | Walltime |
|---|---|---|---|---|
| cpu_sapphire | 45 | 48 core, 512GB RAM | standard | 24 hrs |
| cpu_sapphire_ext | 2 | 48 core, 512GB RAM | standard | 120 hrs |
| cpu_skylake | 52 | 32 core, 384GB RAM | standard | 120 hrs |
| cpu_skylake_ext | 10 | 32 core, 384GB RAM | standard | 240 hrs |
| gpu_a40 | 27 | 48 core, 512GB RAM, 4xA40 | standard | 24 hrs |
| gpu_a40_ext | 4 | 48 core, 512GB RAM, 4xA40 | standard | 120 hrs |
| gpu_a100 | 1 | 48 core, 1024GB RAM, 4xA100 | standard | 24 hrs |
| gpu_h200 | 1 | 64 core, 2048GB RAM, 8xH200 | standard | 24 hrs |
| gpu_v100 | 6 | 32 core, 384GB RAM, 4xV100 | standard | 120 hrs |
| gpu_v100_ext | 1 | 32 core, 384GB RAM, 4xV100 | standard | 240 hrs |

Table 3.3: Legion - Open Access partitions

| Partition | N nodes | Hardware node | Priority | Walltime |
|---|---|---|---|---|
| edu_skylake | 4 | 32 core, 384GB RAM | standard | 24 hrs |
| edu_sapphire | 2 | 48 core, 512GB RAM | standard | 24 hrs |
| edu_v100 | 2 | 32 core, 384GB RAM, 4xV100 | standard | 24 hrs |
| edu_a40 | 2 | 48 core, 512GB RAM, 4xA40 | standard | 24 hrs |

Table 3.4: Legion - EDU partitions

### 3.2.3 EDU partitions

EDU (educational) partitions are reserved for students attending classes which requires access to HPC computational resources. EDU partitions contain a limited subset of open-access nodes as defined in Table 3.4

## 3.3 Cluster networking

Legion deals on three separate networks to operate:

- Management network (1 GbE): to remotely manage individual servers;
- Service network (10 GbE): used by cluster services and for user homes;
- Infiniband network: used for multi-node jobs and high-performance IO.

Management and Service are implemented as standard TCP/IP ethernet networks, with 1 Gb/s and 10 Gb/s bandwidth, respectively. The network topology is linear with all switches connected at the same hierarchical level.

The architecture of the Infiniband network reflects instead the partitioning of Legion into Isola 1 and Isola 2 as two parallely connected fat-tree structures. Isola 1 implements a 100 Gb/s Infiniband EDR fat-tree network, while Isola 2

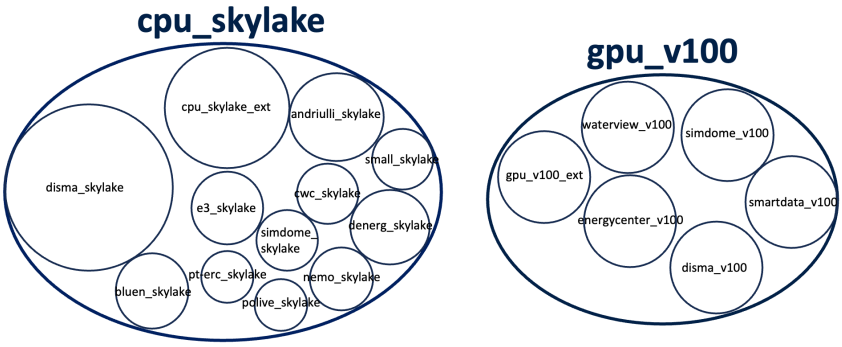



Figure 3.3: Partitions Legion Isola 1

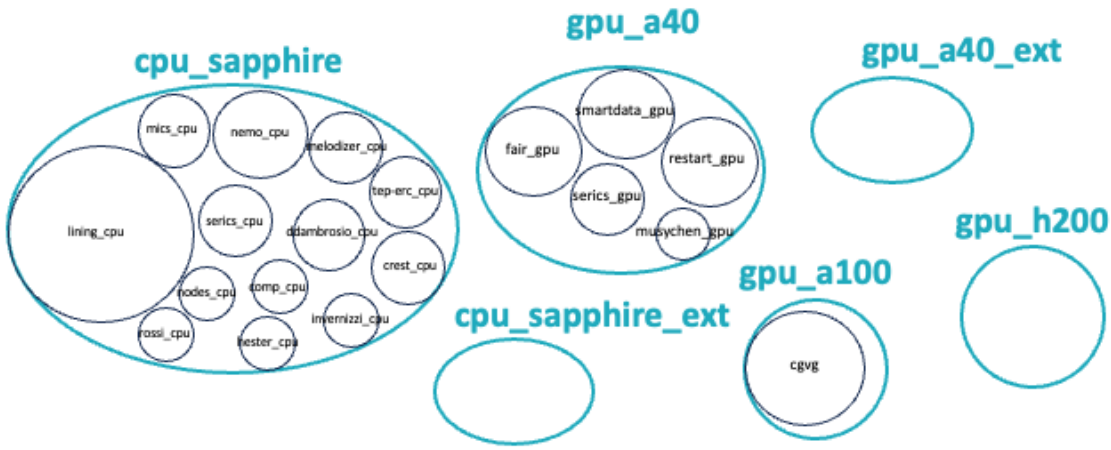



Figure 3.4: Partitions Legion Isola 2

a 200 Gb/s Infiniband HDR fat-tree network. The two root switches are then interconnected with an aggregate of Infiniband EDR links resulting in a 600 Gb/s bandwidth. Although multi-node MPI jobs should not be run between Isola 1 and Isola 2 nodes, a high-bandwidth interconnection between the two guarantees an effective sharing of the storage infrastructure.

These different networks operate in the backend of the cluster and their usage is transparent for users when everything works fine.

## 3.4 Storage systems

The High Performance Computing infrastructure is supported by two complementary storage systems: a high-capacity Network Attached Storage (NAS) to store user homes and data and a lower-capacity BeeGFS parallel filesystem that provides a scratch space for IO intensive workloads that require parallel high-speed operations.

### 3.4.1 Home storage

The Home storage system is implemented with a high-capacity Network Area Storage (NAS) that exports user home folders to login and compute nodes using the NFS protocol over the Service network (10 GbE).

This system provides roughly 1.2 PB of available space to be shared among all users. User quota for the Home storage is defaulted to 2 TB, with possible temporary extensions to be granted upon motivated request. User homes are subject to automatic removal after 6 months from the account expiration.

### 3.4.2 BeeGFS scratch

Currently, we have a new full-flash BeeGFS storage system in production (deployed in January 2026). Our older legacy BeeGFS system (entered production in 2019) has been decommissioned in the early summer of 2026 due to a series of blackout that irremediably compromised its hardware.

**SCRATCH_FLASH** is implemented with a state-of-the art multi-node storage system supported by a fault-tolerant backend network of full-NVMe LUNs. This system provides roughly 800 TB of available space to be shared among all users. User quota for the BeeGFS scratch is defaulted to 10 TB and subject to automatic cleaning of all files older than 30 days. Temporary extensions of the quota may be granted upon motivated request. This filesystem can be accessed via the $SCRATCH_FLASH environment variable.

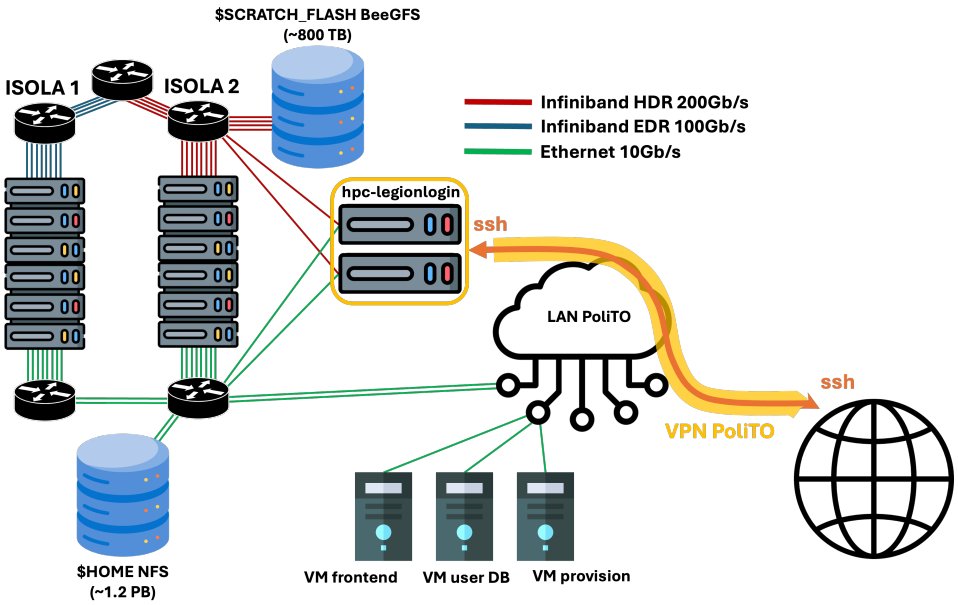



Figure 3.5: Legion overall architecture

## 3.5 Overall architecture

The simplified Legion architecture is reported in Figure 3.5 where the principal components, and how they are connected, are illustrated.

Compute nodes, grouped as Isola 1 and Isola 2, are interconnected via Infiniband network (either 100 Gb/s or 200 Gb/s, colored in blue and red respectively in Figure 3.5) and to the high performance BeeGFS storage, as well as to the login infrastructure. All components are then interconnected by the service network (10 GbE, green in Figure 3.5), which is used by SLURM, NFS sharing and all relevant services. The management infrastructure has been implemented using three different virtual machines (VM), running on a separate high-availability cluster for business continuity reasons, connected to the Legion cluster via a dedicated VLAN. The "VM frontend" hosts the SLURM control daemon and other fundamental services and it is used by HPC@PoliTO personnel to manage the cluster; the "VM user-db" hosts the Free-IPA database with active users; the "VM provision", instead, is used as a platform to update global configurations and deploy new compute nodes.

Users connect to the login infrastructure (hpc-legionlogin.polito.it) via ssh through the internal PoliTO's LAN: if users need to connect from outside PoliTO (or using the internal wi-fi network) they must use our VPN.

# Chapter 4

# HPC@PoliTO file systems

## 4.1 Home directories

HPC@PoliTO users have a home directory (`/home/username`) that is directly accessed upon login. Each user has a default quota of `2 TB` in their home directory, to store relevant data for their projects. Default quotas may be temporarily extended upon motivated request by contacting the HPC@PoliTO staff via email at hpc@polito.it).

As the Home storage system is implemented with a high-capacity NAS (see Section 3.4), IO performances are limited. For this reason simulations should not be run on data stored in the home directory. The home directory should only be used as a "warm" repository for data needed and produced within active projects.

Data stored in home directories will be automatically deleted after 6 months from the expiration of the relative account.

## 4.2 BeeGFS scratch: high performance storage

In addition to their home directories, each user has access to BeeGFS, a high-performance parallel file system. Users can find their BeeGFS space mounted at `$SCRATCH_FLASH`. On `SCRATCH_FLASH` each user has a default quota of `10 TB`. Temporary quota extensions may be granted upon motivated request by contacting the HPC@PoliTO staff via email at hpc@polito.it).

> **Warning:** all files on `$SCRATCH_FLASH` older than 30 days are automatically deleted. This policy is enforced without exceptions or prior notification.

The `SCRATCH_FLASH` BeeGFS space is implemented with a state-of-the-art multi-node full-NVMe storage system, designed to achieve very IO high performances (see Section 3.4). Users should therefore use their BeeGFS scratch space

to store "hot" data needed by current simulations, as well as their immediate outputs (waiting for post-processing).

A legacy `SCRATCH` BeeGFS, first deployed in 2019, has been decommissioned in early summer of 2026 after a series of blackouts that compromised irretrievably its hardware.

> **Warning:** no data protection mechanisms are implemented on the BeeGFS scratch space. Data consistency is never guaranteed. Users should store data on `$SCRATCH_FLASH` only for the strictly necessary time to run simulations and post-process their results.

## 4.3 File transfer between different file systems

Files can easily be copied (using the basic command `cp`) from your home directory (`$HOME`) to your BeeGFS scratch space (`$SCRATCH_FLASH`. To simplify this procedure, two environment variables have been created for all users:

- **$HOME**: the home directory;
- **$SCRATCH_FLASH**: the high-performance full-NVMe BeeGFS file system;

Since different file systems are available to users as different directories, it is possible to move data between them by simply using standard Linux commands (*e.g.* `cp`) or `mv`. For example, to transfer `fileA` from the home directory to the BeeGFS scratch space:

```
cp $HOME/path-to-fileA/fileA $SCRATCH_FLASH/path-to-store/
```

A practical application of file transfer between filesystems, within the context of a SLURM job, can be found in Section 6.2.2.

## 4.4 Best practices

In order to better exploit the features of each file system, users should carefully evaluate where to store their data. While the Home storage (**$HOME**) provides a large and persistent repository for users to store their project data, its read and write performances are not well suited for running simulations on it (*i.e.* jobs should not read or write data on this storage). The Home storage is therefore ideal to store "warm" data, which are not going to be accessed directly during jobs (*e.g.* simulation results that should be stored until the end of the current project).

On the other side, the BeeGFS storage (**$SCRATCH_FLASH**) does not provide a persistent repository for data (files older than 30 days are automatically deleted), but its high performances allow jobs to read and write directly on

it without significant bottlenecks. It is therefore suited to store "hot" data that need to be accessed frequently during jobs, as well as their immediate outputs (*e.g.* raw results that need to be post-processed).

> **Rule of thumb:** use `$HOME` for warm data (project files, post-processed results, code repositories) and `$SCRATCH_FLASH` for hot data (simulation inputs, runtime outputs, scratch files). Never run jobs directly on `$HOME`.

# Chapter 5

# Software ecosystem

The software on the HPC@PoliTO infrastructure can be managed in three different ways:

- **Centralized**: system administrators provide centralized installations of the most frequently used software suites. Users may load the required packages via the ***Environment Modules software***. If a particular software is not present between the available modules, and if it's deemed of interest for the larger community, its installation can be requested by users.
- **Local**: users are allowed to compile and install software in their home directory, without *sudo* privileges. They should specify a location within their home directory during the compilation/installation process. In this case, HPC@PoliTO provides the module ***Spack*** which takes care of handling paths and environment variables of locally installed software: as if they were regular modules.
- **Containers**: users are allowed to execute their own containers, without the need of having *sudo* privileges, using Apptainer.

## 5.1 Environment Modules

The Environment Modules package allows one to dynamically change environment variables during a user session. For users, it appears as if they were "installing" these packages on-demand. In reality, they are changing environment variables to point to centrally installed software packages on the clusters. Using modules is recommended if the needed package and version is available.

In Table 5.1 are reported the most common options to handle modules:

Table 5.1: Environment Modules commands

| Command | Description |
|---|---|
| `module avail` | Shows the modules available on the cluster. |
| `module load {module name}` | Load the specified module. |
| `module show {module name}` | Shows information about the specified module. |
| `module list` | Shows modules load in this user session. |
| `module unload {module name}` | Unload the specified module. |
| `module purge` | Unload all modules in this user session. |

## 5.2 Spack

Spack (https://spack.io/) is a flexible package manager designed specifically for HPC environments. It is particularly valuable for users who need to manage complex software dependencies and maintain reproducible environments across different cluster architectures. The ideal use-case of Spack within the HPC@PoliTO ecosystem is to allow users to locally maintain legacy applications which are no longer provided as a centralized module, as well as niche applications of limited interest for the entire community. Spack simplifies software installation by automatically handling dependencies. Unlike traditional package managers, Spack is non-destructive: installing a new version of software does not break existing installations, allowing multiple configurations to coexist on the same system. Spack uses a specification syntax that allows you to precisely define package requirements, including compiler, and version, as well as optional features.

Unless specifically customized by users, Spack installs packages in a shared folder, so that software can be shared among all users. When a package is loaded, environment variables are directly taken care by Spack.

Installing a new package can be as simple as running the following command:

```
spack install <package name>
```

Previously installed Spack packages can be displayed with the following command:

```
spack find
```

Finally, loading an already available package is as simple as running this command:

```
spack load <package name>
```

The official Spack documentation, as well as baasic tutorials, can be found at the following link: https://spack.readthedocs.io/en/latest/

## 5.3 Apptainer

Apptainer (formerly Singularity) is an open-source container platform specifically designed for HPC environments. It offers a simple, portable, and reproducible way to handle packages and run applications, based on containers rather than local installations. Therefore, Apptainer containers provide application virtualization that ensures consistent workflows across different HPC systems. Finally, contrary to Docker, it does not require root privileges to run and can be used by every user on the cluster. Users may run their containers as simply as writing the following line (within a SLURM script!):

```
apptainer run mycontainer.sif
```

**Warning:** always run containers within SLURM environments (batch or interactive sessions) and never directly on the login nodes.

# Chapter 6

# Jobs submission

Running jobs on HPC@PoliTO's clusters is as simple as running the following command on the shell:

```
sbatch {script name}
```

To understand the meaning of this command and how to actually tailor it to your applications, read through this chapter.

## 6.1 Shared resources

Due to the limited resources available on the cluster and the high number of users, specific mechanisms must be implemented to ensure fair access to computing resources to all users, as well as an efficient use of the infrastructure.

This problem is mainly addressed using a batch scheduler (SLURM), which takes care of collecting job requests from all users and assigns them the necessary computational resources when these are available, based on assigned priorities. More information on SLURM and how priorities are computed can be found in Appendix C.

## 6.2 Submission script

To run jobs on the cluster, users must submit a request to the SLURM scheduler.

> **Warning:** jobs should never be run directly from the shell. It is an abuse of our policies and shows a lack of respect towards your colleagues.

A submission request to SLURM is done via a batch file, which is composed of three main parts:

- job parameters
- application setup

- simulation run

The batch file is a script file executed by the shell. The first line of the file specifies the type of shell in use; in our systems we always use a `bash shell`:

```
#!/bin/bash
```

Some SLURM script examples are reported in Section 6.5.

### 6.2.1 Job parameters

All scheduling parameters passed to SLURM must have the following format:

```
#SBATCH --<parameter>
```

A list of available parameters is reported for reference in Table C.1. Some of them are optional; others may be used alternatively (as SLURM provides many options to fine tune submission request). In this guide, we suggest our preferred configurations while users are always free to adapt their scripts to better suit their workloads. According to HPC@Polito best practices, the following parameters should always be explicitly set:

- `--ntasks`
- `--nodes`
- `--ntasks-per-node`
- `--cpus-per-task`
- `--mem`
- `--time`
- `--partition`
- `--mail-type`
- `--mail-user`
- `--gres=gpu{N-GPUs}`, if the user is submitting a request for a job that requires the use of GPUs

More details about SLURM job parameter are reported in Appendix C.

### 6.2.2 Use the BeeGFS filesystem

Within the SLURM submission script, users can move their data between their home directory and their BeeGFS scratch space, before and after a simulation run. A typical use case, within a job, is the following:

- Move data needed from the current job, from $HOME/path-to-working-dir to $SCRATCH_FLASH/path-to-simulation-dir;
- Change working directory to $SCRATCH_FLASH/path-to-simulation-dir and run your simulation on the much faster BeeGFS storage system (faster IO typically means a faster job run);
- Bring results back from $SCRATCH_FLASH/path-to-simulation-dir to $HOME/path-to-working-dir .

Users may adapt the following example to their needs:

```
[....]
mkdir $SCRATCH_FLASH/path-to-simulation-dir

cp $HOME/path-to-working-dir/* $SCRATCH_FLASH/path-to-simulation-dir

cd $SCRATCH_FLASH/path-to-simulation-dir

[RUN SIMULATION]

rsync $SCRATCH_FLASH/path-to-simulation-dir/* $HOME/path-to-working-dir/
```

### 6.2.3 Simulation run

After setting the job parameters and loading the required software modules (see Chapter 5), the final step is to run the actual simulation. The command used to run a simulation depends on the specific application and how it is parallelized. All commands necessary to run an executable must be included in the script file. Once the scheduler allocates the requested resources, only the commands specified within the script will be run.

For example a generic `MPI-based` parallel application could be run as:

```
mpirun -np $ntasks [EXECUTABLE] -i [input_file] > [output_file]
```

Multiple simulations can be run sequentially within a single job request, specifying multiple execution commands in the submission file. This can be useful when running a series of simulations with different parameters (or configurations). They will be run sequentially following the same order as they have been defined in the submission script, with one starting as soon as the one before finishes.

## 6.3 Submit a job request

After editing the submission file, a job request can be simply submitted to the SLURM scheduler by running the following command:

```
sbatch {script name}
```

where *script name* is the name of the SLURM submission script (provide its path if the script is not in your current work directory).

## 6.4 Check the simulation status

After a job request has been submitted to the SLURM scheduler, it is possible to monitor its progress and/or check its status in the queue. SLURM provides several commands to help you track the state of your simulations, manage your jobs, and gather information about the cluster's resources. It is also possible to setup a mail alerting within the submission script (using the `--mail-user` and `--mail-type`), to be notified when a job starts and finishes.

A list of common SLURM commands to monitor jobs can be found in Table 6.1.

Table 6.1: Common SLURM Commands for Job Monitoring

| **Command** | **Description** |
|---|---|
| `squeue` | Displays the status of jobs in the queue. By default, it shows all jobs, but you can filter by user `squeue -u <username>`, job ID `squeue -j <job_id>` |
| `scontrol show job <job_id>` | Provides detailed information about a specific job |
| `sacct` | Displays accounting information for all your jobs, can be filtered to show the information about a specific job: `sacct -j <job_id>`. |
| `sinfo` | Provides an overview of the cluster's partitions and nodes, including their current availability and status. |
| `scancel <job_id>` | Cancels a job by its ID. Use this command if you need to stop a running or pending job. |

## 6.5 Submission script examples

**EXAMPLE 1:**

The following example submits a parallel MPI job to the `cpu_sapphire` partition, requesting 2 compute nodes (`-N 2`), 48 MPI tasks per node (`--ntasks-per-node=48`), without shared-memory parallelisation (`--cpus-per-task=1`) and without hyperthreading (`--threads-per-core=1`). The executable is run with a total of 96 MPI tasks.

```
#!/usr/bin/env bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=48
#SBATCH --cpus-per-task=1
#SBATCH --threads-per-core=1
#SBATCH --time 0-00:30:00
#SBATCH --partition=cpu_sapphire
#SBATCH --mail-type=ALL
#SBATCH --mail-user=name.surname@polito.it

module purge
module load openmpi/5.0.7_gcc12
module load {my_module} ## change {my_module} with the actual one

mpirun -np 96 my_module_executable [OTHER PARAMETERS]
```

**Variations:**

- **Single node**: set `-N 1` and adjust `mpirun -np` accordingly (*e.g.* 48 for one full Sapphire Rapids node).
- **Pin to specific nodes**: add `#SBATCH --nodelist=compute-X-Y,compute-K-Z` to request particular compute nodes.
- **Exclude specific nodes**: add `#SBATCH --exclude=compute-X-Y,compute-K-Z` to avoid nodes that are known to be problematic or already busy with other jobs from your group.

**EXAMPLE 2:**

The following example submits to the scheduler a hybrid MPI-OpenMP job request, asking for 1 compute node (`-N 1`), running 12 MPI tasks per node (`--ntasks-per-node=12`), with 4 OpenMP threads per task (`--cpus-per-task=4`) and without hyperthreading (`--threads-per-core=1`). The executable is then run using a total of 12 MPI tasks with 4 OpenMP threads per task.

```
#!/usr/bin/env bash
#SBATCH --nodes=1
#SBATCH --ntasks=12
#SBATCH --cpus-per-task=4
#SBATCH --threads-per-core=1
#SBATCH --time 0-00:30:00
#SBATCH --partition=cpu_sapphire
#SBATCH --mail-type=ALL
#SBATCH --mail-user=name.surname@polito.it
```

```
module purge
module load openmpi/5.0.7_gcc12
module load {my_module} ## change {my_module} with the actual one

mpirun -np 12 my_module_executable [OTHER PARAMETERS]
```

**EXAMPLE 3:**

The following example submits to the scheduler an MPI job with GPUs, asking for 1 compute node (`-N 1`), running 4 MPI tasks (`--ntasks-per-node=4`), on 4 GPUs without shared memory parallelization (`--cpus-per-task=1`), without hyperthreading (`--threads-per-core=1`).

```
#!/usr/bin/env bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=1
#SBATCH --threads-per-core=1
#SBATCH --gres=gpu:4
#SBATCH --time 0-00:30:00
#SBATCH --partition=gpu_a40
#SBATCH --mail-type=ALL
#SBATCH --mail-user=name.surname@polito.it
#SBATCH --nodelist=compute-X-Y

module purge
module load openmpi/5.0.7_gcc12
module load {my_module} ## change {my_module} with the actual one

mpirun -np 4 my_module_executable [OTHER PARAMETERS]
```

**EXAMPLE 4:**

The following example shows one way to use the BeeGFS high performance filesystem. It automatically transfers simulation data to $SCRATCH_FLASH, run the parallel simulation and then transfer data back to $HOME (see Section 6.2.2 for further information).

```
#!/usr/bin/env bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=48
#SBATCH --cpus-per-task=1
#SBATCH --threads-per-core=1
#SBATCH --time 0-00:30:00
#SBATCH --partition=cpu_sapphire
#SBATCH --mail-type=ALL
```

```
#SBATCH --mail-user=name.surname@polito.it

module purge
module load openmpi/5.0.7_gcc12
module load {my_module} ## change {my_module} with the actual one

mkdir $SCRATCH_FLASH/path-to-simulation-dir

cp $HOME/path-to-working-dir/* $SCRATCH_FLASH/path-to-simulation-dir

cd $SCRATCH_FLASH/path-to-simulation-dir

mpirun -np 48 my_module_executable [OTHER PARAMETERS]

rsync $SCRATCH_FLASH/path-to-simulation-dir/* $HOME/path-to-working-dir/
```

## 6.6 Interactive sessions

While batch jobs (`sbatch`) are the standard way to run simulations on the cluster, there are situations where an interactive session on a compute node is needed: compiling code, debugging, testing GPU setups, or performing intensive post-processing that should not run on the login nodes.

SLURM provides two mechanisms to request interactive access to compute nodes: direct interactive shells (`srun`) and resource allocation (`salloc`).

### 6.6.1 Quick interactive shell with `srun`

The simplest way to obtain an interactive shell on a compute node is:

```
srun --partition=cpu_sapphire --nodes=1 --ntasks=1 --cpus-per-task=4 \
     --mem=8G --time=1:00:00 --pty bash
```

This command requests resources (in this case: 1 task, 4 CPU cores, 8 GB of RAM, 1 hour walltime on the `cpu_sapphire` partition) and, once they are allocated, opens a `bash` shell directly on the assigned compute node. The session ends when you exit the shell or when the walltime expires.

### 6.6.2 Resource allocation with `salloc`

For more complex workflows — such as running multiple commands, launching MPI processes manually, or testing GPU code — `salloc` is the preferred approach. It allocates resources and returns control to your terminal, from which you can then execute commands on the allocated nodes:

```
salloc --partition=gpu_a40 --nodes=1 --ntasks=1 --cpus-per-task=12 \
       --gres=gpu:1 --mem=32G --time=2:00:00
```

Once the allocation is granted, you can use `srun` within the allocation to execute commands on the compute node, or directly ssh into it:

```
srun --pty bash          # open a shell on the allocated node
nvidia-smi               # verify GPU is visible
python my_training.py    # test your code interactively
```

When done, type `exit` or run `scancel $SLURM_JOB_ID` to release the resources.

> **Key takeaway:** interactive sessions consume cluster resources exactly like batch jobs. Request only what you need and release the allocation as soon as possible. Idle interactive sessions waste resources that could otherwise serve other users.

# Chapter 7

# GPU workflow

Legion provides GPU-accelerated compute nodes across several partitions. This chapter covers the practical aspects of requesting, configuring and using GPUs on the cluster. For hardware specifications and partition details, refer to Chapter 3 (Tables 3.1, 3.2 and 3.3).

## 7.1 GPU resources overview

Table 7.1 summarises the GPU resources available on Legion.

| GPU model | VRAM | GPUs/node | Cores/node | CPUs per GPU | Partitions |
|---|---|---|---|---|---|
| V100 | 32 GB | 4 | 32 | 8 | `gpu_v100`, `gpu_v100_ext` |
| A40 | 48 GB | 4 | 48 | 12 | `gpu_a40`, `gpu_a40_ext` |
| A100 | 80 GB | 4 | 48 | 12 | `gpu_a100` |
| H200 | 141 GB | 8 | 64 | 8 | `gpu_h200` |

Table 7.1: GPU resources on Legion

## 7.2 Requesting GPUs in SLURM

GPUs must be explicitly requested using the `--gres` flag.

> **Warning:** SLURM will *not* allocate GPUs unless the `--gres` flag is set, even if the job is submitted to a GPU partition. Without it, your job will run on a GPU node but will not see any GPU.

```
#SBATCH --gres=gpu:N
```

where `N` is the number of GPUs per node.

### 7.2.1 CPU and memory allocation

When requesting GPUs, the `--ntasks-per-node`, `--cpus-per-task` and `--mem` parameters should be scaled proportionally to the number of GPUs needed. The number of MPI tasks and the amount of memory depends on the application and users should carefully consider these values. For a typical basic use-case (1 MPI task per GPU, RAM= 2 x VRAM), on an A40 node (48 cores, 512 GB RAM, 4 GPUs), requesting 2 GPUs should be paired with:

```
#SBATCH --nodes=1
#SBATCH --partition=gpu_a40
#SBATCH --gres=gpu:2
#SBATCH --ntasks-per-node=2      # 1 task/GPU x 2 GPUs
#SBATCH --cpus-per-task=12      # 12 cores/task
#SBATCH --mem=192G              # 48GB/GPU x 2 x 2 GPUs
```

depending on each application, assigning more cores per each MPI task (–cpus-per-task) may be beneficial for the overall performances.

> **Rule of thumb:** scale CPU and memory requests proportionally to the number of GPUs. Refer to the "CPUs per GPU" column in Table 7.1 for the recommended ratio on each node type. Over-requesting CPUs or memory relative to the number of GPUs blocks resources that other jobs could use on the same node.

## 7.3 GPU environment setup

### 7.3.1 Loading CUDA

CUDA toolkits are provided as environment modules. List available versions with:

```
module avail
```

Load the required version before running your application, for example:

```
module load nvhpc-hpcx-cuda12/25.1
```

Ensure the CUDA version loaded matches the one your application was compiled against. Mismatches between the runtime CUDA version and the compile-time version are a frequent source of errors (see Section 7.5).

### 7.3.2 Verifying GPU allocation

After your job starts (or within an interactive session), verify that the allocated GPUs are visible:

```
nvidia-smi
```

This command should display exactly the number of GPUs requested via `--gres`.

## 7.4 Interactive GPU debugging

Interactive sessions are particularly useful for GPU development and debugging. The recommended workflow is:

1. Allocate a GPU node interactively via `salloc` (see Section 6.6).
2. Verify GPU availability with `nvidia-smi`.
3. Load the required CUDA toolkit and application modules.
4. Run your application interactively to identify issues before submitting a full batch job.

For example, to request 2 A40 GPUs for a 1-hour debugging session:

```
salloc --partition=gpu_a40 --nodes=1 --ntasks-per-node=2 --cpus-per-task=12 \
       --gres=gpu:2 --mem=192G --time=1:00:00
srun --pty bash
module load nvhpc-hpcx-cuda12/25.1
nvidia-smi    # confirm both GPUs are allocated
```

Release the allocation as soon as you are done (`exit` or `scancel $SLURM_JOB_ID`). Interactive GPU sessions are expensive — an idle A40 allocation blocks resources for all other users.

## 7.5 Common GPU issues

**Job runs on a GPU partition but cannot see GPUs.** The `--gres=gpu:N` flag was not set. SLURM does not allocate GPUs by default, even on GPU partitions. Add the flag and resubmit.

**GPU out-of-memory (OOM) errors.** GPU memory (VRAM) is separate from system RAM. A GPU OOM means the model or batch size exceeds the GPU's VRAM (check Table 7.1 for per-card capacity). Reduce batch size, enable checkpointing, or switch to a GPU with more VRAM. Note that a SLURM OOM kill (`OUT_OF_MEMORY` in `sacct`) refers instead to *system* RAM — increase `--mem` in that case.

**CUDA version mismatch.** If your application was compiled with a different CUDA version than the one loaded at runtime, expect crashes or silent numerical errors. Always match the runtime CUDA version to the one used at compile time. Check with `nvcc --version` and compare against your build logs.

**Multi-GPU job uses only one GPU.** Requesting multiple GPUs via `--gres=gpu:N` allocates them, but your application must be explicitly configured to use more than one. Simply requesting 4 GPUs does not make single-GPU code run faster.

**Apptainer container cannot see GPUs.** Add the `--nv` flag when running Apptainer to expose the host's GPU drivers inside the container:

```
apptainer run --nv mycontainer.sif
```

Without `--nv`, the container has no access to NVIDIA devices.

# Appendix A

# Basic Linux usage from command line (CLI)

## A.1 Basic terminal commands

### A.1.1 `man` and `--help` command

To see the full documentation about any command in the terminal, use the command:

```
man <command>
```

To have a quick idea of what the command does and the option of the command, use:

```
<command> --help
```

The former command provides the full documentation, the latter provides a summary of the syntax and of the more commonly used option (usually sufficient for most users).

### A.1.2 `pwd`: your location

To see your current location in the file system, use the command:

```
pwd
```

This command prints the absolute path of the current directory.

### A.1.3 `ls`: See the element inside a directory

To see the elements inside a directory use the command

```
ls [options] <path>
```

If you provide a directory name or a path, the command will show the contents of that specific directory. Use the `-a` option to include hidden files (name starts with `.`), or `-l` to view detailed information about the contents (e.g. dimension, owner, last modified, permissions...). Options can be combined (e.g., `-la`) to display both sets of information. the syntax is: `-la`).

### A.1.4 `cd`: navigate inside the file system

To navigate among folders in the filesystem use the command `cd` (Change Directory). Provide the directory name or its path (paths always start with `/`).

```
cd directory_name
```

To go in the directory before (on the path), use

```
cd ..
```

### A.1.5 Creating and removing directories

To create a new directory use the command:

```
mkdir directory_name
```

If you do not specify a location, the directory will be created at your current position.
To remove a directory, with all the files stored inside, use:

```
rm -r directory_name
```

### A.1.6 `Vim`: file editor

With Vim, you can modify existing files or create new ones:

```
vi file_name
```

If the file_name does not exists the system will automatically create a file with that name in your location (you can open a file in a location different specifying the absolute path of the file). The main commands to use Vim are:

- `i`: insert text (use `esc` to quit inserting mode)
- `:wq`: save and exit
- `:q!`: exit without saving

# Appendix B

# Connection to the clusters via passwordless-ssh

## B.1 Generate key pair from a terminal

After receiving the HPC@PoliTO credentials, to enhance their account security, users should proceed to create their own OpenSSH private/pubic key pair (RSA). More information on RSA cryptography can be found at this link: `https://en.wikipedia.org/wiki/RSA_cryptosystem`.

To generate the private/public key pair you should first open a new shell window in their terminal (Linux and Mac users should use the native terminal while Windows users should use Windows PowerShell). When in a clean terminal window (independently on the actual working directory), you should prompt the following command:

```
ssh-keygen -t rsa
```

The system will propose the path where to save the key (if you change the path, you will need to specify where to find the key each time it is used). The system will then ask for a passphrase to protect the private key. This is a password to protect the private key in the host. If you set a passphrase, every time the key will be used, you will be asked for the passphrase. If you decide not to set a passphrase, leave the field blank.

### B.1.1 Copying the Public Key

After copying the public key from your local host to the cluster, the system will be able, comparing the private and public keys, to recognize you as an authorized user of the system.

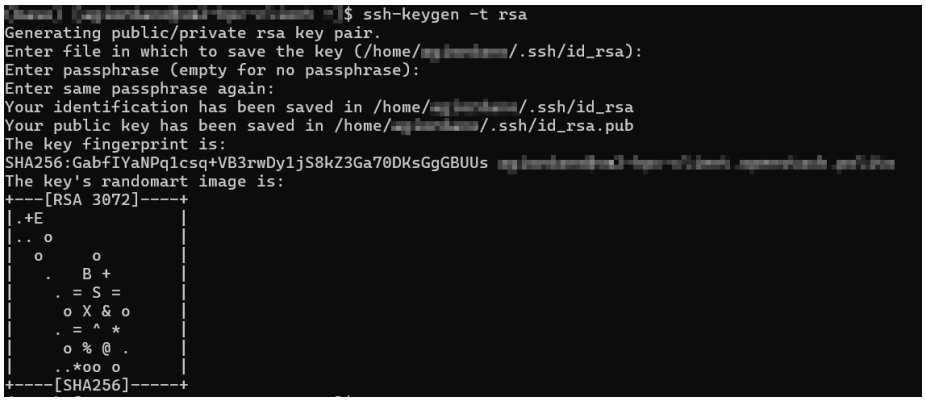



Figure B.1: Output of the command `ssh-keygen`

**From Linux/Unix/OS X systems**

On Unix based systems, there is a built-in command to copy the public key to a given server; this command will take care of all the settings required on the server side:

```
ssh-copy-id [-i <FilePath>/KeyName.pub] username@hpc-legionlogin.polito.it
```

Option `-i` is needed in case you have saved the public key in a non-default path.

**From Windows systems**

If you use the Windows PowerShell terminal, you need to manually copy the public key (`id_rsa.pub`) into the `.ssh` directory on the server. First, create the `.ssh` directory in your home on the cluster:

```
mkdir .ssh
```

Then return to the host shell and use the command `scp` to copy the public key to the cluster:

```
scp <$FilePath>\id_rsa.pub \
    username@hpc-legionlogin.polito.it:/home/username/.ssh/authorized_keys
```

### B.1.2 Login

To check whether the procedure was successful, verify the presence, on the server, of a file named `authorized_keys` within the `.ssh` directory. This file is where the public key is stored; it cannot have another name, since the system will not consider it.

If the file is present, you should be able to log in to the cluster without the need for a password. In case your private key is stored in a non-default directory, you should pass the correct path to the ssh command as:

```
ssh [-i <FilePath>/KeyName] username@hpc-legionlogin.polito.it
```

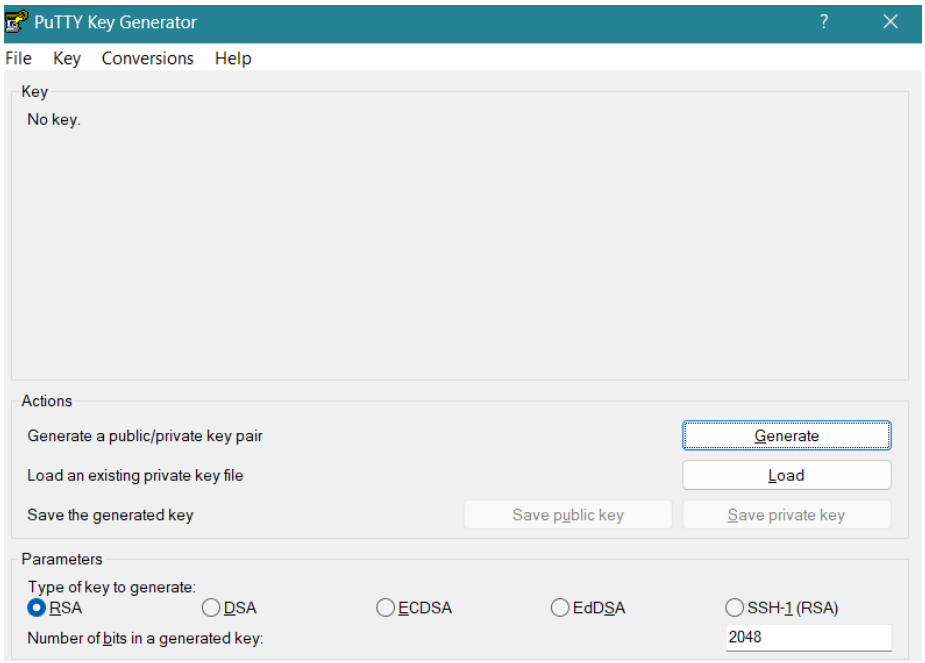



Figure B.2: PuTTYgen homepage

## B.2 Using PuTTY

From a Windows system, it is also possible to use PuTTY software to log in to the cluster.

### B.2.1 Generating a Key with PuTTYgen

To generate a key, you need to install PuTTYgen (a separate application from PuTTY, available at http://www.putty.org, under "Download PuTTY", other binary files). Open PuTTYgen and select the type of key you want to generate (RSA), click on the `generate` button, then proceed by moving the mouse on the blank area that appears on the window to generate the key pair.

Once the key pair has been generated, save the private key on your device (the name and the path where you saved the key are irrelevant) and the public key on the cluster.

### B.2.2 Copying the Public Key

To provide the public key on the cluster, you can either:

- Use the `scp` procedure described above (using a shell terminal).

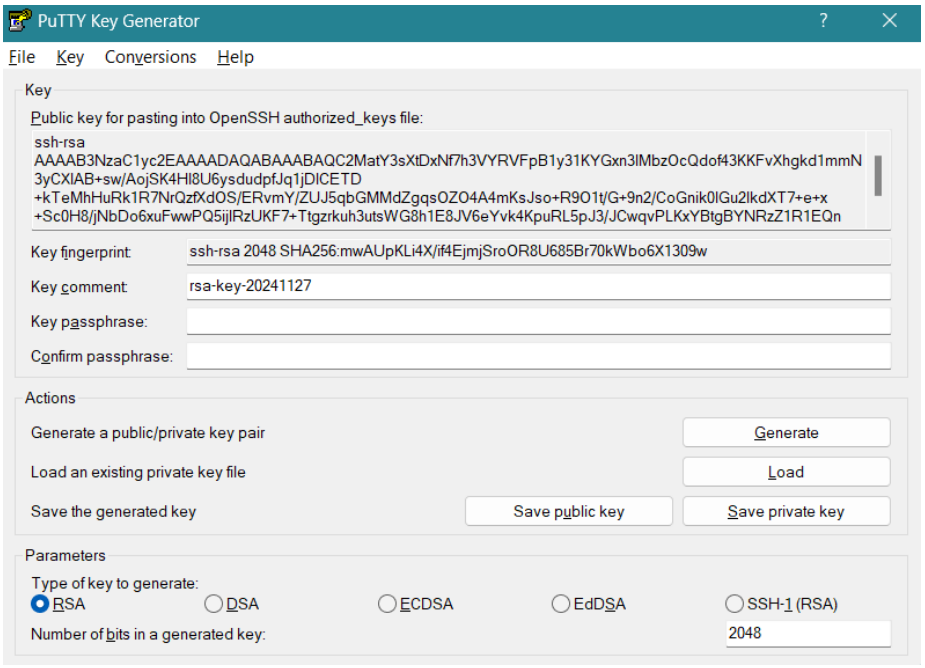



Figure B.3: PuTTY gen has created the key pair

- Copy the public key and paste (if you are using a PuTTY terminal, right click) it into the `authorized_keys` file in the `.ssh` directory on the server.

### B.2.3 Logging in with PuTTY

To log in to the cluster, follow this procedure:

1. Navigate to the section `Connection` → `Auth` → `Credentials`.
2. Add the private key that you created earlier (`key_name.ppk`).
3. Save the session and open it. PuTTY will notify you that it is authenticating using your key.

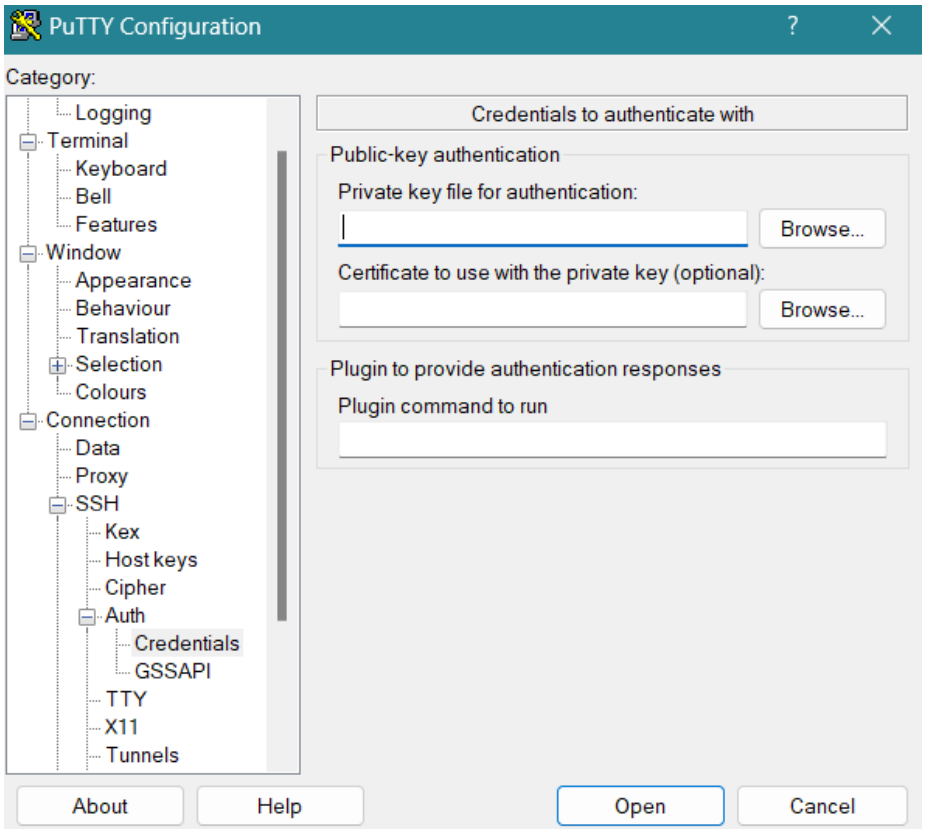



Figure B.4: Set the private key on PuTTY

# Appendix C

# The SLURM scheduler

Table C.1 reports all main scheduling parameters with a brief description of what they do. For a complete reference on SLURM parameters, visit official SLURM webpage at `https://SLURM.schedmd.com/sbatch.html`.

Table C.1: SLURM Job Options and Their Descriptions

| **Command** | **Description** |
|---|---|
| `--ntasks` | Total number of tasks for the job. |
| `--nodes` | Number of nodes that will be used. |
| `--ntasks-per-node` | number of tasks per node, if used with the `--ntasks` option, the `--ntasks` option will take precedence. |
| `--cpus-per-task` | number of CPU per task. |
| `--mem` | Specify the real memory required per node, default units are megabytes. |
| `--time` | Indicates the hard run time limit, which is about the time that processes needs to reach the end. |
| `--mail-user` | Email where the information on task will be sent. |
| `--mail-type` | Events which cause an email notification (e.g., `ALL`). Valid type values are `NONE`, `BEGIN`, `END`, `FAIL`, `REQUEUE`, `ALL`. |
| `--partition` | Indicates the partition where the job has to be scheduled |
| `--gres=gpu:{N.of.gpu}` | Generic resource scheduling, used for specifying the required number of GPU(s) per node. |
| `--nodelist={nodes}` | Request a specific list of hosts. The job will contain all of these hosts and possibly additional hosts as needed to satisfy resource requirements. |
| `--output` | the standard output is redirected to the file name specified, by default both standard output and standard error are directed to the same file. |

Table C.1: SLURM Job Options and Their Descriptions

| Command | Description |
|---|---|
| `--error` | instruct Slurm to connect the batch script's standard error directly to the file name specified. |
| `--workdir={directory}` | If present, cause the execution of task using the {directory} as working directory (the path of {directory} can be specified as full path or relative path). |
| `--mem-per-cpu` | Minimum memory required per allocated CPU, default units are megabytes (default=1000). |
| `--exclude` | Explicitly exclude certain nodes from the resources granted to the job. |
| `--constraint` | Nodes have features assigned to them and users can specify which of these features are required by their job using this option, for example `--constraint="gpu"`. Available features can be shown in the output section `scontrol show node`. |

## C.1 Priority calculation

Job scheduling with SLURM is based on a multi-factor priority system: each pending job is assigned a numeric score, computed as a weighted sum of several factors, including partition tier, fair-share, QOS, job size, queue age and GPU resource requests. The partition tier is by far the dominant component on Legion, ensuring that jobs on restricted partitions are always prioritized over open-access ones. Among the remaining factors, fair-share (which tracks each group recent resource usage over a 30-day window) is the primary lever users can influence through their submission behavior.

> **Rule of thumb:** the partition tier outweighs all other priority components combined. Within the same partition tier, fair-share is the strongest lever — spread your submissions over time to maintain a healthy fair-share factor.

A comprehensive description of the priority formula, its weights, and worked examples can be found in the *HPC@PoliTO Job Scheduling Policy* document, available at `www.hpc.polito.it/docs/hpcpolito_scheduling.pdf`.

# Appendix D

# Restricted partitions

| Partition | N nodes | Hardware node | Priority | Walltime |
|---|---|---|---|---|
| lining_cpu | 23 | 48 core, 512GB RAM | highest | N/A |
| nemo_cpu | 3 | 48 core, 512GB RAM | highest | N/A |
| melodizer_cpu | 2 | 48 core, 512GB RAM | highest | N/A |
| mics_cpu | 2 | 48 core, 512GB RAM | highest | N/A |
| serics_cpu | 2 | 48 core, 512GB RAM | highest | N/A |
| teperc_cpu | 2 | 48 core, 512GB RAM | highest | N/A |
| ddambrosio_cpu | 2 | 48 core, 512GB RAM | highest | N/A |
| crest_cpu | 2 | 48 core, 512GB RAM | highest | N/A |
| nodes_cpu | 1 | 48 core, 512GB RAM | highest | N/A |
| comp_cpu | 1 | 48 core, 512GB RAM | highest | N/A |
| hester_cpu | 1 | 48 core, 512GB RAM | highest | N/A |
| invernizzi_cpu | 1 | 48 core, 512GB RAM | highest | N/A |
| rossi_cpu | 1 | 96 core, 512GB RAM | highest | N/A |
| fair_gpu | 6 | 48 core, 512GB RAM | highest | N/A |
| restart_gpu | 6 | 48 core, 512GB RAM, 4xA40 | highest | N/A |
| smartdata_gpu | 6 | 48 core, 512GB RAM, 4xA40 | highest | N/A |
| serics_gpu | 3 | 48 core, 512GB RAM, 4xA40 | highest | N/A |
| musychen_gpu | 2 | 48 core, 512GB RAM, 4xA40 | highest | N/A |
| cgvg_gpu | 1 | 48 core, 1024GB RAM, 4xA100 | highest | N/A |
| disma_skylake | 16 | 32 core, 384GB RAM | highest | N/A |
| denerg_skylake | 6 | 32 core, 384GB RAM | highest | N/A |
| bluen_skylake | 6 | 32 core, 384GB RAM | highest | N/A |
| e3_skylake | 3 | 32 core, 384GB RAM | highest | N/A |
| simdome_skylake | 2 | 32 core, 384GB RAM | highest | N/A |
| small_skylake | 2 | 32 core, 384GB RAM | highest | N/A |
| cwc_skylake | 2 | 32 core, 384GB RAM | highest | N/A |
| andriulli_skylake | 2 | 32 core, 384GB RAM | highest | N/A |
| nemo_skylake | 2 | 32 core, 384GB RAM | highest | N/A |
| pt-erc_skylake | 1 | 32 core, 384GB RAM | highest | N/A |
| polive_skylake | 1 | 32 core, 384GB RAM | highest | N/A |
| waterview_v100 | 1 | 32 core, 384GB RAM, 4xV100 | highest | N/A |
| simdome_v100 | 1 | 32 core, 384GB RAM, 4xV100 | highest | N/A |
| disma_v100 | 1 | 32 core, 384GB RAM, 4xV100 | highest | N/A |
| smartdata_v100 | 1 | 32 core, 384GB RAM, 4xV100 | highest | N/A |
| energycenter_v100 | 1 | 32 core, 384GB RAM, 4xV100 | highest | N/A |

Table D.1: Legion - Restricted access partitions